

Bases de datos - UD3. Ejercicios. Diseño Físico y Edición de Datos

[Descargar estos ejercicios](#)

Índice

▼ Creación de tablas

- ☒ [Ejercicio 1](#)
- ☒ [Ejercicio 2](#)
- ☒ [Ejercicio 3](#)
- ☒ [Ejercicio 4](#)
- ☒ [Ejercicio 5](#)
- ☒ [Ejercicio 6](#)
- ☒ [Ejercicio 7](#)

▪ Edición de datos

▼ Ejercicios finales

- ☒ [Ejercicio ??](#)
- ☒ [Ejercicio 2](#)

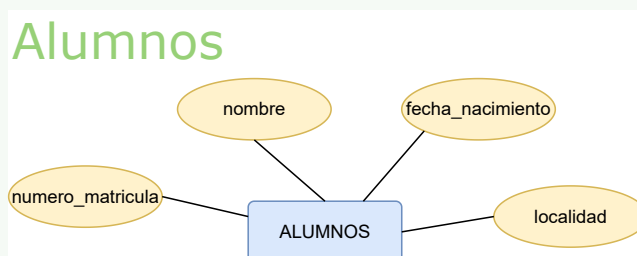
Creación de tablas

Se valorará además de que la solución de los ejercicios sea correcta, las tabulaciones, los comentarios en el código, nombres de elementos (tablas y campos) en minúscula y la claridad del código en general.

☒ Ejercicio 1

Alumnos

A partir del esquema conceptual (modelo entidad-relación) siguiente



Se obtiene el siguiente esquema lógico (modelo relacional):

ALUMNOS (numero_matricula, nombre , fecha_nacimiento , direccion , localidad)

Escribe la instrucción en SQL para crear la tabla ALUMNOS teniendo en cuenta las siguientes restricciones:

- Restricciones del esquema ER:
 - Clave primaria: numero_matricula
- Restricciones del diccionario de datos:
 - El atributo nombre no puede tomar valores nulos

💡 Solución

```
CREATE DATABASE ejercicios;

CREATE TABLE ALUMNOS (
  numero_matricula INT(6) PRIMARY KEY,
  nombre VARCHAR(15) NOT NULL,
  fecha_nacimiento DATE,
  direccion VARCHAR(30),
  localidad VARCHAR(50)
);
```

✅ Ejercicio 2

Alumnos

Intenta crear de nuevo la tabla “ALUMNOS” de la actividad anterior. ¿Qué ocurre?

💡 Solución

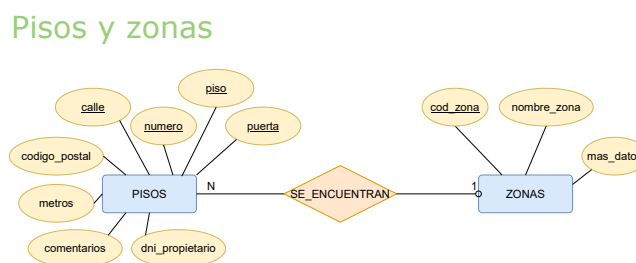
Table 'ALUMNOS' already exists

SQL devuelve un error, indicando que la tabla ya existe y no se puede volver a crear.

✅ Ejercicio 3

Pisos y zonas

A partir del esquema conceptual (modelo entidad-relación) siguiente



Escribe las instrucciones en SQL para crear las tablas obtenidas en el esquema lógico (del modelo relacional).

Primero crearemos la tabla ZONAS, ya que PISOS hace referencia a dicha tabla.

Recuerda que otra opción sería:

- Crear primero la tabla PISOS sin clave ajena

- Crear la tabla ZONAS
- Añadir la restricción de clave ajena a la tabla PISOS.

ZONAS (cod_zona <entero 2>, nombre_zona <caracter 20>, mas_datos <caracter 50>)

Restricciones del esquema ER:

- Clave primaria: cod_zona
 - Observa que la clave primaria está formada por el atributo cod_zona, y que aunque no lo especifiquemos como NOT NULL automáticamente al formar parte de la clave el SGBD coloca NOT NULL en este atributo.
 - Como la clave primaria está formada por una única columna podemos poner la restricción a continuación de la definición de dicha columna o al final de la tabla. En este caso la pondremos junto a la definición de la columna.

PISOS (calle <caracter 30>, numero <entero 3>, piso <entero 2>, puerta <caracter 1>, codigo_postal <entero 5>, metros <entero 5>, comentarios <caracter 60>, **cod_zona* <entero 2>**, dni_propietario <caracter 9>)

Restricciones del esquema ER:

- Clave primaria: calle, numero, piso, puerta. (Nombra a esta restricción como pk_viv)
 - Observa que la clave primaria está formada por los atributos calle, número, piso y puerta y que aunque no los especifiquemos como NOT NULL automáticamente al formar parte de la clave coloca NOT NULL en esos atributos.
 - Recuerda que como la clave primaria está formada por un grupo de columnas hemos de definirla al final de la tabla.
- Clave ajena: cod_zona → ZONAS
- De momento no incluimos las restricciones por ON DELETE o por ON UPDATE. ES decir, de momento no se especifica qué debe hacer la base de datos automáticamente cuando se elimine o actualice un registro relacionado en una clave foránea.

Solución

```
CREATE TABLE ZONAS (
  cod_zona INT(2) PRIMARY KEY,
  nombre_zona VARCHAR(20),
  mas_datos VARCHAR(50)
);

CREATE TABLE PISOS (
  calle VARCHAR(30),
  numero INT(3),
  piso INT(2),
  puerta CHAR(1),
  codigo_postal INT(5),
  metros INT(5),
  comentarios VARCHAR(60),
  cod_zona INT(2),
  dni_propietario CHAR(9),
  CONSTRAINT pk_viv PRIMARY KEY (calle, numero, piso, puerta),
  FOREIGN KEY (cod_zona) REFERENCES ZONAS(cod_zona)
);
```

✓ Ejercicio 4

Partiendo del esquema lógico, crea la tabla de **proveedores** de nuestra comunidad autónoma. La estructura de la tabla es la siguiente:

proveedores(*cod_prov* , *localidad* , *nombre* , *provincia*)

El diccionario de datos (sólo con restricciones de valor) es:

Tabla proveedores			
CAMPO	TIPO	LONGITUD	CARACTERÍSTICAS
cod_prov	Cadena	3	Clave Primaria No Nulo
localidad	Cadena	100	Nulo
nombre	Cadena	100	No nulo Único Clave Alternativa
provincia	Enumerado		'Alicante', 'Valencia', 'Castellón'

💡 Solución

```
CREATE TABLE proveedores (  
  cod_prov    VARCHAR(3) PRIMARY KEY,  
  localidad  VARCHAR(100) DEFAULT NULL,  
  nombre     VARCHAR(100) NOT NULL UNIQUE,  
  provincia  ENUM('Alicante', 'Valencia', 'Castellón')  
);
```

✓ Ejercicio 5

En XAMPP-MySQL mediante la utilidad **mysql.exe** ejecuta las instrucciones anteriores.

Entra a **phpMyAdmin** y comprueba con el asistente visual la estructura de la tabla y crea otra llamada **proveedores2** con la misma información con la herramienta gráfica.

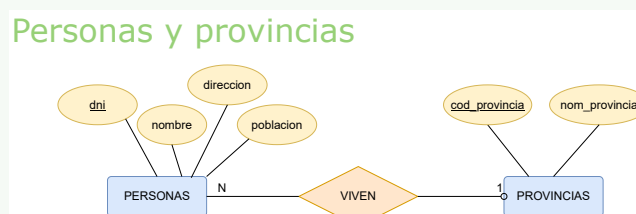
Pulsando en la pestaña *Insertar*, añade registros a la tabla **proveedores2**.

Comprueba con el botón *Examinar* los datos introducidos. Comprobarás que aparece un botón *Editar* y otro *Borrar* que afecta a cada registro.

✓ Ejercicio 6

Personas y provincias

Del esquema conceptual (del modelo entidad-relación) siguiente:



Escribe las instrucciones en SQL para crear las tablas obtenidas en el esquema lógico (del modelo relacional).

La tabla PERSONAS contiene datos sobre las personas de una comunidad, mientras que la tabla PROVINCIAS contiene el código y el nombre de cada provincia, se relacionan por el atributo codprovin.

En primer lugar crearemos la tabla PROVINCIAS y, después la tabla PERSONAS, ya que PERSONAS referencia a PROVINCIAS. Si creamos primero la tabla PERSONAS y la tabla PROVINCIAS no está creada, MySQL emitirá un mensaje de error.

Recuerda que otra opción sería:

- Crear primero la tabla PERSONAS sin clave ajena
- Crear la tabla PROVINCIAS
- Añadir la restricción de clave ajena a la tabla PERSONAS

Cuando crees las tablas, hazlo sin asignar nombre a las restricciones de clave primaria ni tampoco a las de clave ajena.

```
PROVINCIAS (cod_provincia <entero 2>, nom_provincia <caracter 25>)
```

Restricciones del esquema ER:

- Clave primaria: cod_provincia

```
PERSONAS (dni <entero 8>, nombre <caracter 15>, direccion <caracter 25>, poblacion <caracter 20>, cod_provin* <entero 2>)
```

Restricciones del esquema ER:

- Clave primaria: DNI.
- Clave ajena: cod_provin → PROVINCIAS.
 - Como la clave ajena está formada por un único atributo se puede definir tanto a continuación de la definición de la columna cod_provin como al final de la definición de la tabla. En este caso, lo haremos a continuación de la definición de la columna y no especificaremos ninguna política ante borrados para asegurar la integridad referencial.
 - De momento no incluimos las restricciones por ON DELETE o por ON UPDATE

Solución

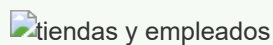
```
CREATE TABLE PROVINCIAS (
    cod_provincia INT(2) PRIMARY KEY,
    nom_provincia VARCHAR(25)
);

CREATE TABLE PERSONAS (
    dni INT(8) PRIMARY KEY,
    nombre VARCHAR(15),
    direccion VARCHAR(25),
    poblacion VARCHAR(20),
    cod_provin INT(2),
    FOREIGN KEY (cod_provin) REFERENCES PROVINCIAS(cod_provincia)
);
```

✓ Ejercicio 7

Tiendas y empleados

Del esquema conceptual (del modelo entidad-relación) siguiente:



Escribe las instrucciones en SQL para crear las tablas obtenidas en el esquema lógico (del modelo relacional).

Primero crearemos la tabla TIENDAS, ya que EMPLEADOS hace referencia a dicha tabla.

TIENDAS (codigo <entero 2>, nombre <caracter 25>)

Restricciones del esquema ER:

- Clave primaria: codigo

EMPLEADOS (nombre <caracter 25>, edad <entero 3>, **cod_tienda* <entero 2>**)

Restricciones del esquema ER:

- Clave primaria: nombre
- Clave ajena: cod_tienda → TIENDAS

Restricciones del diccionario de datos:

- Al borrar la fila asociada a una tienda deseamos que se eliminen automáticamente las filas con claves ajenas que la referencien, es decir, que se eliminen todos los empleados de esa tienda.
- **Verificación de condiciones: edad comprendida entre 18 y 65.**

💡 Solución 1

```
CREATE TABLE TIENDAS (
    codigo INT(2) PRIMARY KEY,
    nombre VARCHAR(25)
);

CREATE TABLE EMPLEADOS (
    nombre VARCHAR(25) PRIMARY KEY,
    edad INT(3) CHECK (edad BETWEEN 18 AND 65),
    cod_tienda INT(2),
    FOREIGN KEY (cod_tienda) REFERENCES TIENDAS(codigo)
    ON DELETE CASCADE
);
```

Otra opción sería:

- Crear primero la tabla EMPLEADOS sin clave ajena
- Crear la tabla TIENDAS
- Añadir la restricción de clave ajena a la tabla EMPLEADOS.

Solución 2

```
CREATE TABLE EMPLEADOS (
    nombre VARCHAR(25) PRIMARY KEY,
    edad INT(3) CHECK (edad BETWEEN 18 AND 65),
    cod_tienda INT(2)
);

CREATE TABLE TIENDAS (
    codigo INT(2) PRIMARY KEY,
    nombre VARCHAR(25)
);

ALTER TABLE EMPLEADOS
ADD FOREIGN KEY (cod_tienda) REFERENCES TIENDAS(codigo)
ON DELETE CASCADE;
```

Edición de datos

Ejercicios finales

Ejercicio ??

Atletismo

Haz un script de creación de la base de datos ATLETISMO (atletismo.sql) a partir del modelo lógico-relacional.

Los tipos de datos de los distintos campos serán los que consideres convenientes, siempre que la solución adoptada sea coherente.

CLUBS(codigo, nombre, ciudad)

ATLETAS(nif, nombre, apellidos)

COMPETICIONES(codigo, descripcion, ciudad, fechaEvento)

ACTIVIDADES_DEPORTIVAS(codigo, descripcion)

FECHAS(fechaAlta)

PARTICIPAR(club, competicion)

CF: club → CLUBS

CF: competicion → COMPETICIONES

PRACTICAR(atleta, deporte, plusmarca)

CF: atleta → ATLETAS

CF: deporte → ACTIVIDADES_DEPORTIVAS

PERTENECER(atleta, fecha, club, fechaBaja)

CF: atleta → ATLETAS

CF: fecha → FECHAS

CF: club → CLUBS

COMPETIR(competicion, atleta, deporte, marca)

CF: competicion → COMPETICIONES

CF: atleta → ATLETAS

CF: deporte → ACTIVIDADES_DEPORTIVAS

[atletismo.sql](#)

✓ Ejercicio 2

StarTrek

Haz un script de creación de la base de datos STARTREK (startrek.sql) a partir del modelo lógico-relacional.

Los tipos de datos de los distintos campos serán los que consideres convenientes, siempre que la solución adoptada sea coherente.

ACTORES(idActor, nombre, nacionalidad, fechaNacimiento)

PERSONAJES(nombre, raza, graduacionMilitar, actor, capMilitar)

CF: actor → ACTORES

CF: capMilitar → PERSONAJES

PELÍCULAS(titulo, director, añoLanzamiento, protagonista)

CF: protagonista → PERSONAJES

CAPÍTULOS(titulo, temporada, ordenRodaje, fechaEmision1)

PLANETAS(codigo, nombre, galaxia)

NAVES(codigo, nombre, tripulacion)

APARECER(personaje, pelicula)

CF: personaje → PERSONAJES

CF: pelicula → PELÍCULAS

PARTICIPAR(capitulo, personaje)

CF: capitulo → CAPÍTULOS

CF: personaje → PERSONAJES

VISITAR(capitulo, planeta, nave, problema)

CF: capitulo → CAPÍTULOS

CF: planeta → PLANETAS

CF: nave → NAVES

[startrek.sql](#)